DASHBOARD PROJECT

Structured summary — organized by Front-end, Back-end & Database,
with an urgency sticker (1–3) on every point.

Legend — urgency stickers

U3 U3 - Critical

U2 U2 - Medium

U1 U1 - Low

*Urgency reflects how central the point is to passing the project's core validation (account flow, services/widgets, dashboard CRUD, timer, and the mandatory **GET /about.json** on port 8080 via docker-compose).*

Project overview

Build a highly customizable, full-stack dashboard (a browser new-tab-style control panel) that lets an authenticated user subscribe to services (GitHub, Steam, Weather, RSS, Market data, Maps, etc.), add configurable widget instances for those services, and see their data refreshed automatically by a timer. Group sizing rule: for X students, support at least (1 + X) services and (3 × X) widgets in total.

FRONT-END

Client interface, dashboard UX, widget & auth screens

Registration & login screens (email sign-up or OIDC), with confirmation step before access	U3
Main Dashboard page: grid/canvas that renders all added widget instances	U3
"Add widget" flow: pick a widget type → configure its params → set refresh rate → confirm	U3
Reconfigure an existing widget instance from the dashboard	U2
Move (reposition/drag) a widget instance on the dashboard	U2
Delete a widget instance from the dashboard	U2
Service subscription UI: browse available services, enter credentials or trigger OAuth link	U3
Widget rendering components per service (weather, GitHub, Steam, RSS, market data, maps, etc.)	U3

Responsive layout + accessibility (keyboard nav, contrast, ARIA) across devices	U2
User profile page (manage own credentials / linked accounts)	U2
Admin section UI for managing site users	U1
Client-side validation of widget configuration forms (string/integer params)	U2

BACK-END

API, auth/authz, services logic, widgets, timer, /about.json endpoint

User management module: registration endpoint + confirmation flow	U3
Authentication service: credentials (username+password) and/or OIDC	U3
Authorization layer: OAuth 2.0 flow for third-party services	U3
Link a third-party OAuth account to an existing platform user (not a new one)	U3
Service subscription API: store which services a user activated + their credentials/tokens	U3
Per-service data fetchers (GitHub, Steam, Weather, RSS, Market data, Maps, etc.) via external APIs	U3
Widget catalog logic: each service exposes widgets with typed, configurable params	U3
Timer / scheduler: receives each widget instance's refresh rate and triggers data refresh	U3
REST endpoint GET /about.json (server MUST run on port 8080) exposing client host, server time, services & widgets	U3
Input validation & sanitation on every widget/service configuration	U3
Secure handling & storage of credentials/tokens (encryption, no leakage in logs/responses)	U3
Admin API endpoints for user management	U1
Dockerized server service wired into docker-compose.yml (build + up must succeed)	U3

DATABASE

Persistence: users, identities, subscriptions, widget instances

Users table: account info, hashed credentials, confirmation/verification status	U3
Linked identities table: OIDC/OAuth external accounts mapped to a platform user	U3
Services table: catalog of available services (name, auth type required)	U2
Widgets catalog table: widget name/identifier per service + description	U2
Widget params schema table: param name + type (string/integer) per widget	U2
User service subscriptions table: user ↔ service, with stored credentials/OAuth tokens	U3
Widget instances table: owner, widget type, chosen params/config, refresh rate, position on dashboard	U3
Sessions / auth tokens store (login sessions, refresh tokens)	U2
Admin/audit table for user management actions (optional but useful)	U1

Cross-cutting constraints (apply to all layers)

docker-compose.yml at repo root; server service must run on port 8080 and answer GET /about.json	U3
A widget with no configurable parameters is invalid and won't be counted	U3
Two instances of the same widget with different configs must show distinct data simultaneously	U3
README.md: features/services/widgets list, install & build steps, usage, tech stack & architecture overview	U3
All source files delivered except binaries/temp/obj files; bonus files go in a bonus/ directory	U2
Justify technology choices professionally; avoid over-engineering; secure-by-design (validate inputs, protect credentials)	U2